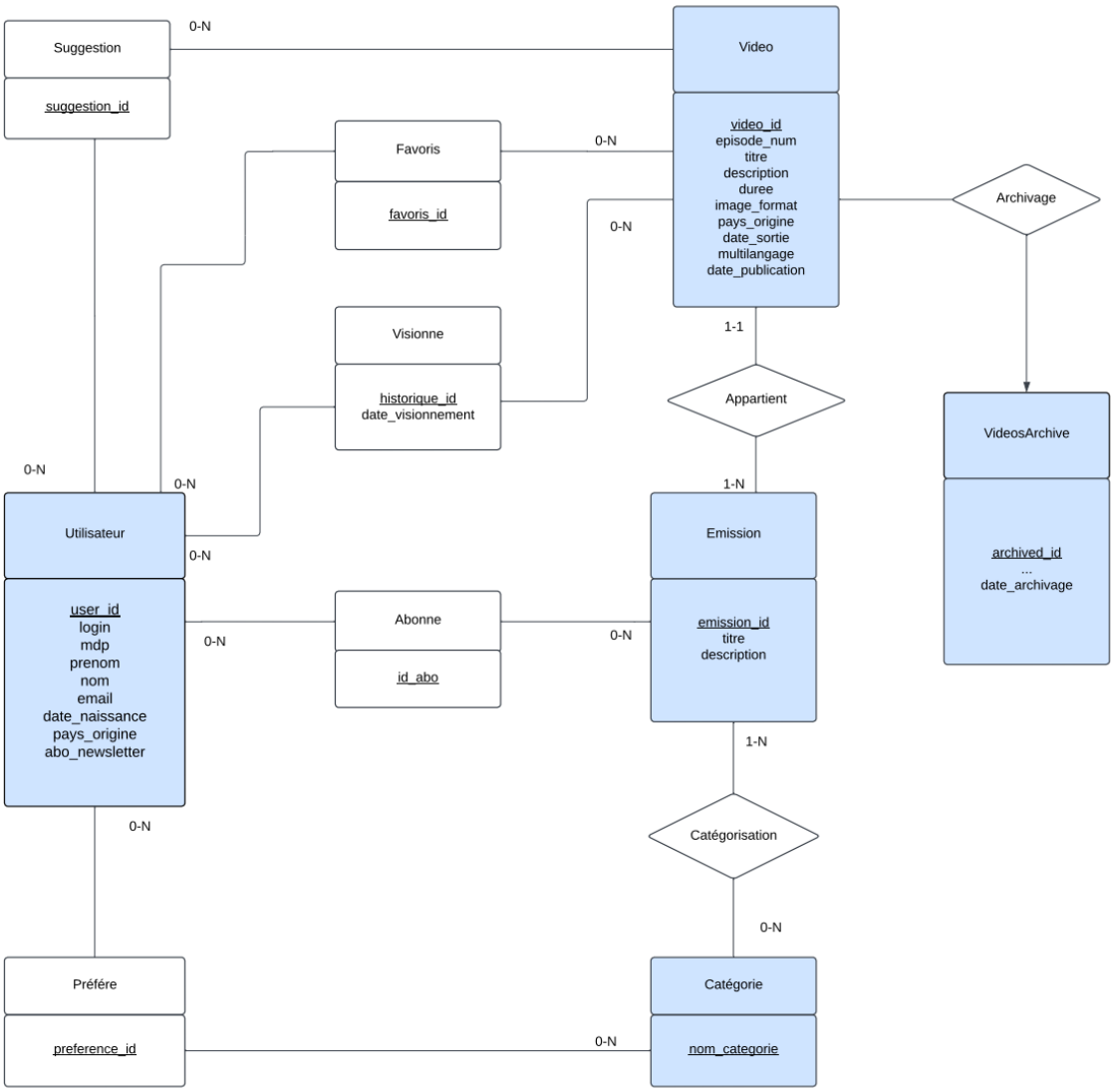
PROJET BDD2

Ahmet Berk ATES

Sommaire

PROJET BDD2 1
Modèle Entité Association 2
Remarques 2
Modèle relationnel 4
Commentaire sur tables 4
Les contraintes 5
Justification des index par rapport aux 5 requêtes 7
Explication de l'utilisation de Mockaroo et des ajustements apportés 9
Mon approche pour les tests de déclencheurs 10

Modèle Entité Association



La cardinalité peut être souvent 0 ou N. Autrement dit, pour donner un exemple : un utilisateur peut ne peut avoir regarder de vidéos et une vidéo peut avoir aucun vu. Ou une émission, peut ne pas avoir de vidéos, d'épisodes de telle sorte qu'une émission est annoncé mais elle n'a pas encore d'épisodes. Le cas d'archivage est plus délicat, une vidéo est archivée mais la relation est selon moi dans un sens unidirectionnel. On ne peut retracer vers une vidéo qui existe toujours, cela créerait une redondance et lenteur dans le site. Au lieu de cela je garde les informations dans une table d'archive avec en plus la date d'archivage qui me semble être un champ pertinent.

Remarques

Au début du projet, j'ai hésité entre créer une table distincte pour les épisodes et une table pour les vidéos. Cependant, j'ai réalisé que les informations nécessaires pour les vidéos et les épisodes étaient presque équivalentes. J'ai donc décidé de combiner les deux dans une seule

table nommée Video. Dans cette table, le titre d'une vidéo correspond au titre d'un épisode. Cette approche simplifie la gestion des données, car chaque vidéo représente un épisode, et la plupart des informations nécessaires sont directement disponibles dans cette table. Cela m'évite d'avoir à effectuer des jointures supplémentaires pour récupérer les informations sur les vidéos.

De plus en ce qui concerne, je tiens a précisé que j'ai utilisé la date de sortie pour évaluer si une vidéo est nouvelle, tandis que j'ai utilisé la date de publication sur le site pour gérer l'archivage de certaines vidéos. De plus, on peut annoncer, publier la sortie d'un film qui va sortir.

J'ai fait des insertions avec quelques dates de sortie et de visionnement plus tardive que le 8 janvier car je ne sais pas quand cela sera corrigé. Aujourd'hui le 8 janvier, mes requêtes, mes procédures de newsletter, de recommandations fonctionnent bien mais si la correction se fait dans plusieurs semaines, alors il pourrait peut-être y avoir des problèmes. Dans ce cas-je vous prie d'augmenter le nombre de jours dans les requêtes et procédures.

Pour mes insertions, il y a un nombre limité de pays mais je ne les restreints pas dans les contraintes, je suppose que la base de données recevra bien des nationalités grâce a l'implémentation du site.

La gestion d'archivage : j'ai implémenté un trigger qui empêche l'archivage de la vidéo avant les 7 jours (/moreTriggers/contrainteSeptJours.sql) en plus de la surpression de la vidéo qui résulte en l'archivage de la vidéo (triggers/triggerArchive.sql). J'ai également une procédure qui faudrait périodiquement lancer pour archiver des vidéos qui datent.

J'ai préparé plusieurs contraintes dynamiques en plus pour les contraintes dynamiques que je cite dans la liste de contraintes. Cependant, je ne suis pas certain de la nécessité de vérifier coté serveur, par exemple si la date de publication est avant la date de sortie, car cela est faisable par des vérifications moins couteuses autre part. Plus de contraintes pourraient être présentes. Celles que j'ai fait en plus de ceux demandés explicitement se trouvent dans './moreTriggers'

J'ai favorisé l'utilisation des id au lieu des doubles clefs, que j'aurais pu faire par exemple pour UserPreference, pour faciliter les procédures qui utiliseraient des séquences.

Les insertions que j'ai fait avec l'IA étaient difficiles à restreindre avec certaines de mes contraintes comme la date de sortie qui doit être avant ou en même temps que la date de publication. Je considère ces contraintes plutôt plus utiles pour des insertions prochaines. Je les ai testés avec des données de test. Les données insérées initialement ne respectent pas tout le temps toutes ces dernières.

En revanche pour l'archive je ne me suis pas servis de l'IA, j'utilises l'archives pour mes tests, j'inserts puis supprime les données de tests, j'ai décidé qu'il n'était pas nécessaire de faire d'insertion initiale avec l'IA.

Modèle relationnel

Utilisateur (user_id : Integer, login : String, mdp : String, prenom : String, nom : String, email : String, date_naissance : Date, abo_newsletter : Integer, pays_origine : String)

Categorie (nom_categorie : String)

Emission (emission_id : Integer, titre : String, description : String, #nom_categorie : String)

Video (video_id : Integer, #emission_id : Integer, episode_num : Integer, titre : String, description : String, duree : Integer, image_format : String, pays_origine : String, date_sortie : Date, multilangage : Integer, date_publication : Date)

UserPreference (preference_id : Integer, #user_id : Integer, #nom_categorie : String)

UserAbonnement (abo_id : Integer, #user_id : Integer, #emission_id : Integer)

Favoris (favoris_id : Integer, #user_id : Integer, #video_id : Integer)

Historique (historique_id : Integer, #user_id : Integer, #video_id : Integer, date_visionnement: Date)

VideoSuggestion (suggestion_id : Integer, #user_id : Integer, #video_id : Integer, date_creation: Date)

VideosArchive (archived_id : Integer, #video_id : Integer, emission_id : Integer, episode_num Integer, titre : String, description : String, duree : Integer, image_format : String, pays_origine: String, date_sortie : Date, multilangage : Integer, date_archivage : Date, date_publication : Date)

Commentaire sur les tables

Les champs sont systématiquement non nuls.

En plus des contraintes d'âge (contrainteNaissance.sql) et de courriel valide, l'unicité du pseudonyme est importante car plusieurs personnes peuvent porter le même prenom et nom de familles. J'ai représenté les booléens par (0,1).

J'ai restreint les catégories à six catégories.

Une émission a un titre unique et appartient à une seule catégorie, elle aurait pu appartenir à plusieurs mais j'ai décidé comme cela.

Une vidéo, ne peut pas représenter plusieurs épisodes avec le même numéro pour une même émission et ne peut pas avoir le même titre avec un autre épisodes de la même émission. De même, elle peut être la vidéo que d'une seule émission. Je suppose que la continuité des épisodes n'est pas importante. Sa date de sortie (contrainte_date_publication.sql) ne peut être après sa date de publication, une contrainte dynamique.

Chaque préférence d'utilisateur a un id, j'aurais pu mettre user_id et nom_categorie comme clé mais je préfère cette approche pour pouvoir utiliser des séquences et faire des procédures facilement.

Un utilisateur peut s'inscrire à une émission mais pas plusieurs fois à un même émission donc unique couple user_id, emission_id dans UserAbonnement.

Un utilisateur peut avoir plusieurs vidéos en favoris mais pas deux fois la même d'où l'unique couple user_id, video_id dans Favoris.

Dans l'historique, on stocke l'historique des utilisateurs, chaque ajout par défaut est ajoute avec la date de visionnement, une vidéo peut être revue plusieurs fois donc pas d'unique couple cette fois. Une contrainte dynamique est également présente (contrainte_date_vision.sql) pour modifier avant insertion si la date de visionnement est une date à venir, elle le remet à présent.

Une vidéo dans la table vidéo ne peut être archivée avant d'être publiée pour sept jours, cela est rassuré avec une contrainte dynamique (contrainteSeptJours.sql)

Les contraintes

Utilisateur :

Il se peut que deux personnes aient le même prénom et nom mais leur login doit être unique.

User_id : clé primaire
Login : unique et non nul
Mdp, prénom, nom : non nul
Email : doit contenir @
Date_naissance : non nul (statique) et au moins 2 ans (dynamique, contrainteNaissance)
Abo_newsletter : par défaut 0, 1 ou 0
Pays_origine : non nul

Catégories :

Nom_categorie : doit être dans la liste suivante : ('comedy', 'drama', 'kdrama', 'horror', 'romantic', 'sci-fi')

Emissions :

Emission_id : clé primaire
Titre : non nul et unique
Description : non nul
Nom_categorie : clé étrangère
Unique (titre)

Vidéos :

Video_id : clé primaire
Emission_id : clé étrangère
Episode_num, titre, : non nul
Description : par défaut ‘ ’
Durée : durée > 0 et non nul
Image_format : doit être dans la liste suivante ('8k', '4k', 'uhd', 'hd')
Pays_origine : non nul
Date_sortie : non nul
Multilangage : 0 ou 1, 0 par default
Date publication : par défaut CURRENT_TIMESTAMP
Contrainte dynamique (contrainteSeptJours) qui empêche les suppressions quand la publication date de moins de sept jours.
Contrainte dynamique (contrainte_date_publication) : date publication >= Date sortie
Contrainte dynamique, une vidéo est archivée quand elle est supprimée
Unique (emission_id, episode_num)
Unique (emission_id, titre)

J’utilises une procédure pour archiver les vidéos publiés, il y a 20 ans. En théorie, je voudrais la faire tourner périodiquement mais je n’ai pas les droits donc j’ai laissé en commentaire, comment je pensais pouvoir exécuter celle-ci périodiquement.

UserPreferences :

Preference_id : clé primaire
User_id : clé étrangère
Nom_categorie non nul
Unique (user_id, nom_categorie)

UserAbonnement :

Abo_id : clé primaire
User_id : clé étrangère
Emission_id : clé étrangère
Unique (user_id, emission_id)

Favoris :

Favoris_id : clé primaire
User_id : clé étrangère
Video_id : clé étrangère
Unique (user_id, video_id)
Limite de 300 favoris par un utilisateur (dynamique)

Historique :

Historique_id : clé primaire
User_id : clé étrangère
Video_id : clé étrangère
Date_visionnement : par défaut current_timestamp
Contrainte_date_vision (dynamique), à l'insertion si la date de visionnement est au futur alors on le met à la valeur par défaut.

VideosArchive

Archived_id : clé primaire
Video_id, emission_id, episode_num, titre, description, durée, image_format, Pays_origine, Date_sortie : non nul
Multilangage : 1 ou 0 et non nul
Date_archivage : par défaut current_timestamp
Date_publicaton : non nul

VideosSuggestion :

Suggestion_id : clé primaire
User_id : clé étrangère
Video_id : clé étrangère
Date_creation : par défaut current_timestamp
Contrainte dynamique de spam

Justification des index par rapport aux 5 requêtes

Pour optimiser les performances des requêtes présentes dans le fichier requetes.sql, voici une justification des index utilisés et leur pertinence par rapport à chaque requête.

Requête 1 : Sélection des catégories les plus visionnées au cours des 14 derniers jours

- **Index utiles :**
  - idx_historique_date_visionnement sur Historique(date_visionnement)
  - idx_historique_video_id sur Historique(video_id)
  - idx_videos_video_id sur Video(video_id)
  - idx_videos_emission_id sur Video(emission_id)
  - idx_emissions_emission_id sur Emission(emission_id)
- **Justification :**
  - L'index idx_historique_date_visionnement permet de filtrer rapidement les enregistrements de l'historique des visionnages des 14 derniers jours.

- Les index idx_historique_video_id, idx_videos_video_id, et idx_videos_emission_id permettent de joindre efficacement les tables Historique, Video, et Emission.
- L'index idx_emissions_emission_id permet de joindre efficacement la table Emission.

Requête 2 : Comptage des abonnements, favoris et vidéos regardées par utilisateur

- **Index utiles :**
  - idx_userabonnement_user_id sur UserAbonnement (User_id)
  - idx_favoris_user_id sur Favoris (User_id)
  - idx_historique_user_id sur Historique (User_id)
- **Justification :**
  - Les idx_userabonnement_user_id, idx_favoris_user_id, et idx_historique_user_id permettent de joindre efficacement les tables Utilisateur, UserAbonnement, Favoris, Historique sur la colonne user_id.
  - Ces index améliorent les performances des comptages et des jointures nécessaires pour obtenir les abonnements, favoris et vidéos regardées par utilisateur.

Requête 3 : Comptage des visionnages par pays d'origine

- **Index utiles :**
  - idx_historique_video_id sur Historique(video_id)
  - idx_historique_user_id sur Historique (User_id)
  - idx_users_pays_origine sur Utilisateur(pays_origine)
- **Justification :**
  - Les index idx_historique_video_id, et idx_historique_user_id permettent de joindre efficacement les tables Video, Historique, et Utilisateur.
  - L'index idx_users_pays_origine permet de filtrer rapidement les utilisateurs par pays d'origine.
  - Ces index améliorent les performances des comptages et des jointures nécessaires pour obtenir les visionnages par pays d'origine.

Requête 4 : Sélection des vidéos populaires par émission

- **Index utiles :**
  - idx_historique_video_id sur Historique(video_id)

- idx_videos_emission_id sur Video(emission_id)

- **Justification :**
  - Les index idx_historique_video_id et idx_videos_emission_id permettent de joindre efficacement les tables Historique et Video.
  - Ces index améliorent les performances des comptages et des jointures nécessaires pour obtenir les vidéos populaires par émission.

Requête 5 : Comptage des paires de vidéos regardées par les mêmes utilisateurs

- **Index utiles :**
  - idx_historique_user_id sur Historique (User_id)
  - idx_historique_video_id sur Historique(video_id)
- **Justification :**
  - Les index idx_historique_User_id et idx_historique_video_id permettent de joindre efficacement les enregistrements de la table Historique sur les colonnes User_id et video_id.
  - Ces index améliorent les performances des comptages et des jointures nécessaires pour obtenir les paires de vidéos regardées par les mêmes utilisateurs.

Explication de l'utilisation de Mockaroo et des ajustements apportés

Pour générer les données de test nécessaires à mon projet, j'ai utilisé l'outil en ligne Mockaroo. Cet outil m'a permis de créer des ensembles de données réalistes et variés pour les différentes tables de ma base de données. Voici comment j'ai procédé et les ajustements que j'ai apportés :

1. **Génération des données avec Mockaroo :**
   - J'ai utilisé Mockaroo pour générer des données de test pour les tables Utilisateurs, Categories, Emissions, Videos, Favoris, UserPreferences, Abonnements, VideoSuggestions et Historique.
   - Mockaroo m'a permis de spécifier les types de données et les contraintes pour chaque colonne, ce qui a facilité la création de données réalistes et cohérentes.
2. **Correction des erreurs de contraintes d'unicité :**
   - Après avoir inséré les données générées par Mockaroo, j'ai rencontré des erreurs de contraintes d'unicité pour certaines colonnes.
   - J'ai corrigé ces erreurs en ajustant manuellement les valeurs des colonnes concernées pour garantir que chaque valeur soit unique, conformément aux contraintes définies dans la base de données.

3. **Insertion des données dans la table Historique :**

    - Pour la table Historique, j'ai inséré des données spécifiques en tenant compte des contraintes de date.
    - J'ai ajouté un peu d'enregistrements avec des dates de visionnement vers la fin janvier pour essayer de m'assurer que certaines vidéos soient populaires au moment de la correction du projet. De plus, j'ai inséré ces enregistrements avant d'activer le trigger suivant, afin de garantir qu'il y ait des vidéos populaires lors de la correction. S'il n'y en a plus, s'il vous plait, veuillez ajuster l'intervalle de temps la ou j'ai précisé dans mon fichier de procédures.

Mon approche pour les tests de déclencheurs

Isolation des tests :

Chaque test que j'ai réalisé est isolé en utilisant des variables locales pour stocker les données de test. Cela signifie que chaque test dispose de son propre ensemble de données, ce qui évite toute interférence avec d'autres tests. Les données de test sont insérées, vérifiées, puis supprimées dans le même bloc PL/SQL. Cette méthode garantit que les tests sont indépendants les uns des autres et que les résultats sont fiables.

Nettoyage des données de test :

Après chaque test, je m'assure de supprimer les données de test des tables concernées. Cette étape est cruciale pour maintenir un environnement de test propre. En supprimant les données de test après chaque exécution, je m'assure que les tests suivants ne sont pas affectés par les données résiduelles des tests précédents. Cela permet de garantir que chaque test est exécuté dans un environnement propre et sans interférence.

Utilisation de `DBMS_OUTPUT.PUT_LINE` :

J'utilise les messages `DBMS_OUTPUT.PUT_LINE` pour afficher des informations sur les différentes étapes du test. Ces messages incluent les données insérées, les résultats obtenus et les actions de nettoyage. Cette pratique facilite le débogage et la vérification des résultats des tests. En affichant ces informations, je peux suivre le déroulement de chaque test et identifier rapidement les éventuels problèmes.

En suivant cette approche, je m'assure que chaque test est autonome et que les données de test sont nettoyées après chaque exécution. Cela garantit un environnement de test propre et fiable, où les résultats des tests sont précis et reproductibles. Cette méthode me permet de maintenir un haut niveau de qualité et de fiabilité dans mes tests, en minimisant les risques d'interférence et en facilitant le débogage.

En plus de ces derniers, quand j'ai créé des nouvelles contraintes, j'ai aussi revérifié que les tests des autres contraintes passaient et rectifié les commandes si nécessaires. Du fait que je

n’ai pas directement implémenté toutes les contraintes puis tous les tests, j’ai dû corriger des insertions dans les données de tests.